

AGENTDAG RESEARCH BIBLE

AgentDAG Bible

DAG 기반 멀티에이전트 오류 전파 억제 연구

2026-05-16 · 팀 회의 자료

PART 1

결론 요약

연구 질문 · 핵심 기여점



연구 질문 & 기여점

DAG 중간 노드에 평가 게이트를 달면 오류 전파율이 얼마나 줄어드는가?
Partial Backpropagation이 게이트 단독 대비 추가로 얼마나 줄이는가?

기존 연구

- 어떻게 DAG를 실행하는가
- 오류를 어떻게 탐지하는가
- 최종 출력 기준 평가

우리가 추가하는 것

- 게이트 + Partial Backprop의 억제 효과
- Fault Injection 실험으로 정량화
- 정형/비정형 태스크 타입별 분석

#	기여	근거
C1	DAG 중간 게이트가 오류전파율을 정량적으로 줄임을 실증	Exp09: Vanilla vs Option1
C2	Partial Backpropagation이 게이트 단독 대비 추가 억제 효과 검증	Exp09: Option1 vs Option2
C3	정형/비정형 태스크 타입에 따라 오류 전파 패턴이 다를 것을 발견	분리 집계 분석

PART 2

DAG 기술 개요

DAG 기술 기초 · 시스템 아키텍처 · 실험 그래프



DAG 기술 기초

DAG란

Directed Acyclic Graph — 노드(작업 단위) + 방향 엣지(데이터 의존성) + 비순환 보장.
실행 순서는 위상 정렬(Topological Sort)로 결정론적으로 결정.

Linear: A → B → C
Parallel: A → B, A → C (B·C 동시)
Fan-in: B → D, C → D (D는 B·C 완료 후)
Diamond: A→B→D, A→C→D (Fan-out+Fan-in)

7대 표준	정의
위상 정렬	선행 노드 항상 먼저 실행
Critical Path	전체 실행 시간 결정 최장 경로
Dataflow 모델	데이터 가용성이 실행 트리거
멱등성	동일 입력 → 동일 출력 (재시도 안전)
체크포인팅	노드 완료 시 상태 영속 저장
Fan-out/Fan-in	분배 → 합산 패턴
비순환 보장	정의 시점 DFS로 사이클 차단

구성 요소	역할
Orchestrator	위상 정렬 → 실행 순서 결정, 노드 상태 추적, 재시도·HITL 라우팅
Worker	개별 노드 작업 수행 (LLM 호출)
State Store	노드별 입출력·상태 영속 저장
Evaluator	노드 출력 품질 판정 → PASS / FAIL / AMBIGUOUS

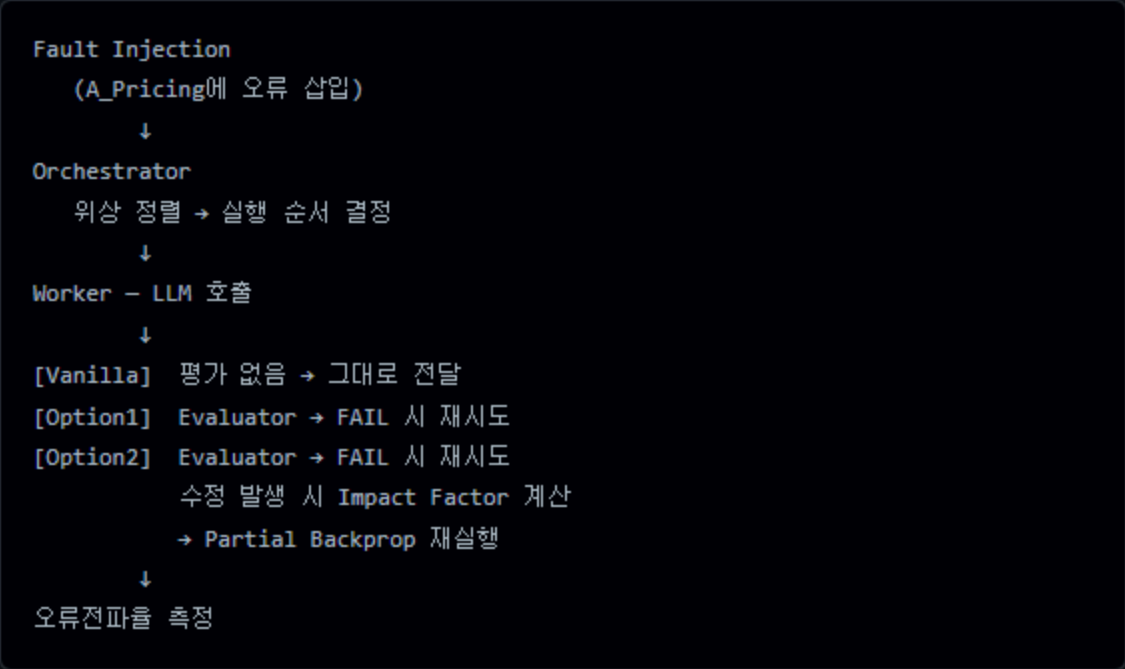
시스템 아키텍처

코드 구조

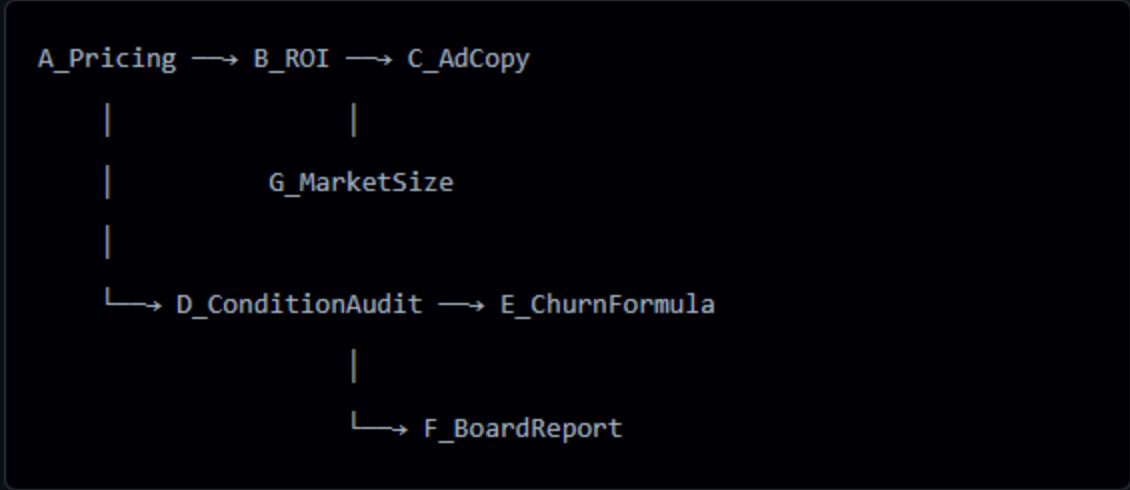
```
paper/src/
├─ config.py      - 하이퍼파라미터
├─ models.py      - NodeState, GraphState
├─ graphs.py      - 7-node SaaS Pricing
├─ evaluator.py   - PASS/FAIL/AMBIGUOUS
├─ orchestrator.py - DAG 실행, 위상 정렬,
│                   Partial Backprop
└─ evaluators/    - Exp03~08 변형
```

역할	모델	이유
Worker	gpt-4o-mini	비용 효율
Evaluator	gpt-4o	정확도 우선
Embedding	text-embedding-3-small	경량

실행 흐름 (Exp09)



실험 그래프 — 7-node SaaS Pricing



노드	타입	역할
A_Pricing	정형	3-tier 구독 가격 설계 (Pro=2.5×Basic)
B_ROI	정형	A 가격 기반 연간 ROI 계산
C_AdCopy	비정형	B의 ROI% 인용 광고 문구 작성
D_ConditionAudit	비정형	계약 조건 검토
E_ChurnFormula	정형	이탈률 공식 계산
F_BoardReport	비정형	경영진 보고서
G_MarketSize	비정형	시장 규모 분석

PART 3

DAG 최신 기술 정보

연구 지형도 · Gap Map · EPR 선행 연구 · 2026 생태계



연구 지형도

분야	키워드	연결점
구조	DAG Multi-Agent Orchestration	핵심 실험 환경
평가	Error Propagation Measurement	핵심 측정 지표
신뢰성	Fault Injection & Reliability	실험 방법론
개입	Human-in-the-Loop (HITL)	AMBIGUOUS + Impact Factor
시각화	Agent Observability & Tracing	논문 Figure

순위	분야	점수
1	Multi-Agent Orchestration / DAG	9.1
2	Error Propagation Measurement	8.7
3	LLM-as-Judge / Agent Evaluation	8.4
4	Error Recovery & Resilience	7.8
5	Human-in-the-Loop (HITL)	7.5

* 점수 기준: 논문 기여 가능성(4) + 차별화 여지(3) + 현재 관심도(3)

분야별 최신 연구 & 한계

분야	논문	연도	핵심
DAG	DynTaskMAS (2503.07675)	ICAPS 2025	DAG 자동 생성 + 비동기 병렬, 실행 21~33% 단축
	Scheduler-Theoretic (2604.11378)	2026	형식적 스케줄링 이론 (실증 없음)
	Why Do MAS Fail? / MAST (2503.13657)	2025	14개 실패 유형, 1,600+ 트레이스
EPR	AgentEval (2604.23581)	2026	루트코즈 vs 전파 실패 구분. 전체 실패의 63%가 upstream 전파
	AgentProp-Bench (2604.16706)	2026	S1→S2→S3 단계별 전파율. 기준값 ≈0.62
	MAS-FIRE (2602.19843)	2026	15개 결함 유형 × 3가지 비침습 주입 방식
HITL	HULA (2411.12924)	ICSE 2025	개발자 HITL 프레임워크, 플랜 승인률 82%

핵심 한계: 중간 노드 품질 검증 없음 / 최종 출력만 검증 / 오류 전파 범위의 형식적 보장 없음

연구 공백 지도 (Gap Map)

현재 연구 집중 영역	공백 — 우리가 파고들 곳
DAG 실행 스케줄링	게이트 유무·백프로п 유무가 오류전파율에 미치는 단계적 효과
오류 전파 측정 (단일 파이프라인)	DAG 중간 게이트가 전파율을 구조적으로 줄이는지 실증
단일 실행 트레이싱	정형/비정형 태스크 타입에 따른 전파 패턴 차이
HITL 개입 메커니즘	게이트 + 백프로п 조합의 비용 대비 전파 억제 효율

기존 연구가 다루는 것

- "어떻게 DAG를 실행하는가"
- "오류를 어떻게 탐지하는가"

우리가 추가하는 것

- DAG 중간 게이트가 오류 전파를 얼마나 줄이는가
- Partial Backprop의 추가 억제 효과 정량화

EPR 선행 연구 — AgentEval & AgentProp-Bench

AGENTEVAL (ARXIV:2604.23581)

에이전트 실행을 평가 DAG로 형식화, 노드별 LLM Judge 부착

- 루트코즈 실패 **36.8%** / 전파 실패 **63.2%**
- 평균 전파 체인 길이: **2.1** 노드
- FDRec 0.89 vs 0.41 (E2E 대비 2.17×)
- 인간 전문가 합의 $\kappa = 0.84$

→ 루트코즈 vs 전파 실패 구분으로 게이트 효과 직접 측정 가능

AGENTPROP-BENCH (ARXIV:2604.16706)

파라미터 수준 fault injection 후 3단계 지표로 전파율 분해

S1 = 주입 적용됨
S2 = 하위 실행에 observable 변화
S3 = 최종 출력 오류

$EPR = P[S3|S1] \approx 0.62$
(범위: 0.46 ~ 0.73)

- 거절률·회복률은 독립 능력 ($p=0.126$)
- 전파율 모델별 편차 큼 (0.43 ~ 0.97)

→ S1→S3 추적으로 게이트 개입 단계 정량화

EPR 선행 연구 — MAS-FIRE & Sherlock

MAS-FIRE (ARXIV:2602.19843)

15개 결함 유형 × 3가지 비침습 주입 방식으로 MAS 강건성 평가

Of = 결함 인식 후 방어 행동 유발 비율
Lf = 지역 복구 성공률
Sf = 지역 복구 → 전역 성공 비율
Lf - Sf 갭 = 잔류 전파 효과

- 반복 페루프: 선형 대비 결함 **40%+ 무력화**
- 아키텍처 토폴로지 $\geq$ 모델 크기

→ Option2 백프롬의 Lf/Sf 차이로 보완 측정

SHERLOCK (ARXIV:2511.00330)

반사실적 분석으로 취약 노드 식별 → 선택적 검증기 부착

노드 취약성 순서:
말단 노드 > 초기 노드 > 중간 노드

말단: 복구 경로 없음
초기: 오류가 전체 파급
중간: 하위 노드가 보정 가능

- Fan-in $\uparrow$ → 오류 증폭 가능성 $\uparrow$
- 정확도 **+18.3%** / 실행시간 **-48.7%**

→ A_Pricing(초기 노드) 주입 위치 이론적 뒷받침

선행 연구 종합 — 실험 설계 원칙

설계 결정	근거 논문	구체 수치
루트 노드(A_Pricing)에 주입	Sherlock — 초기 노드 = 전체 파급	15K+ 트레이스 실증
루트코즈 vs 전파 실패 구분	AgentEval	실패의 63% 가 전파 실패
단계별 전파율(S1→S3) 측정	AgentProp-Bench	기준값 $\approx$ 0.62
정형/비정형 분리 집계	MAS-FIRE	아키텍처별 편차 40%+
Fan-in 노드 중점 관찰	Sherlock	Fan-in ↑ → 오류 증폭

주입 위치

A_Pricing (초기 노드)

EPR 기준값

Vanilla $\approx$ 0.62 예상

중점 관찰 노드

D_ConditionAudit, F_BoardReport (Fan-in)

2026 AI Agent 생태계

프로토콜	방향	2026 상태
MCP	Agent → Tools/Data	월 9,700만 SDK 다운로드, 서버 18,000+
A2A	Agent ↔ Agent	v1.0 GA, 150+ 조직 지원
LangGraph	프레임워크 오케스트레이션	프로덕션 표준 (Supervisor 패턴)

항목	현재 상태	갭
LangGraph	사용 중	interrupt() / Command(resume=...) 미적용
체크포인트	자체 구현	AsyncPostgresSaver 교체 고려
MCP	미적용	Evaluator를 MCP 서버로 노출 → 외부 재사용 가능
HITL	자체 구현	LangGraph interrupt() 패턴으로 표준화 가능
A2A	미적용	단일 배포 → 현재 불필요

PART 4

실험 옵션

오늘 회의에서 결정할 4가지 항목



실험 형식 — Vanilla vs Option1 vs Option2

	OPTION A	OPTION B	OPTION C
Vanilla	게이트 X, 백프롬 X	게이트 X, 백프롬 X	게이트 X, 백프롬 X
Option1	게이트 O, 백프롬 X	단순 게이트 O, 백프롬 X	게이트 O, 백프롬 X
Option2	게이트 O, 백프롬 O	엄격 게이트 O, 백프롬 X	게이트 O, 적응형 재실행 O
비교 축	게이트 유무 × 백프롬 유무	게이트 엄격도	재실행 방식
논문 기여도	★★★★★	★★★★☆	★★★★☆
실험 실현성	★★★★★	★★★★☆	★★★★☆
결과 설득력	★★★★★	★★★★☆	★★★★☆
코드 재활용	★★★★☆	★★★★★	★★★★★
종합	19/20	13/20	17/20

권장: Option A — 게이트 유무·백프롬 유무를 단계적으로 분리해 기여점 C1·C2를 직접 증명.

→ 회의 결정: _____

오류전파율 측정 방식

	방식 A — 구조적	방식 B — 단계적	방식 C — 루트코즈	방식 D — 이중계층
수식	오류 하위 노드 수 / 전체 하위 노드 수	$P[S3 S1] \approx 0.62$ S1→S2→S3 분해	전파 실패 수 / 전체 실패 수	Of / Lf / Sf 이중 계층
출처	자체 설계	AgentProp-Bench	AgentEval	MAS-FIRE
논문 정합성	★★★★☆	★★★★★	★★★★★	★★★★☆
구현 용이성	★★★★★	★★★★☆	★★★★☆	★★★★☆
DAG 구조 활용	★★★★☆	★★★★☆	★★★★★	★★★★☆
전파 경로 추적	★★★★☆	★★★★☆	★★★★★	★★★★☆
종합	14/20	15/20	17/20	11/20

권장: 방식 C — AgentEval 선행 연구와 직접 비교, DAG 의존성 최대 활용.
차선: 방식 A + B 병행 — 구현 단순 + 단계별 추적, 논문 Figure 두 가지 확보.

→ 회의 결정: _____

평가항목 — 정형 / 비정형

	정형만	혼합 (분리 집계)
대상 노드	A_Pricing, B_ROI, E_ChurnFormula	7-node 전체
오류 판정	수치 위반 (명확)	정형: 수치 위반 / 비정형: LLM Judge
논문 기여도	★★★★☆	★★★★★
실험 실현성	★★★★★	★★★★☆
일반화 주장	약함	강함
추가 설계 부담	없음	비정형 판정 기준 1회 설계 필요
종합	13/20	18/20

권장: 혼합 — 기여점 C3(정형/비정형 전파 패턴 차이) 추가 확보. SaaS 그래프 그대로 사용.

기본 형태 (이미 확정)
단방향 DAG — 7-node SaaS Pricing 그래프 유지.

→ 회의 결정: _____

다음 단계 체크리스트

오늘 회의

- [회의] 실험형식 확정 — Option A / B / C
- [회의] 오류전파율 측정 방식 확정 — A / B / C / D
- [회의] 평가항목 (정형/비정형) 확정
- [확정] 기본 형태 — 단방향 DAG (7-node SaaS)

이후 구현

- [미완] Fault injection 레이어 구현 (A_Pricing 오류 주입)
- [미완] 오류 판정 기준 코드화 정형: 수치 위반 / 비정형: LLM Judge
- [미완] Exp09 — Vanilla vs Option1 vs Option2
- [미완] 결과 시각화 — 오류전파율 비교 Figure
- [예정] Exp10 — 신재용 데이터셋 일반화 검증